Tagado — Product & Build Spec (v1)

Nov 10, 2026 · @Ninja

For Cursor: read this whole spec first, then propose an implementation plan following section 19, one phase at a time. Stack: React (Vite, TypeScript) frontend, Python (FastAPI) backend, Postgres. No AI libraries or AI keys anywhere.

1. The product in one page

Tagado tracks money a business is owed and follows up with each client until a human confirms the money arrived. Claude writes the follow-ups through MCP; Tagado stores, schedules, sends, reacts to client responses and remembers each client's payment history.

Customer: small Indian B2B service businesses (agencies, studios, IT consultancies) whose founder already pays for Claude. They invoice in Zoho, Tally or Excel. Tagado does not create GST invoices; it works on top of any invoice.

Problem: invoices get paid late, chasing is awkward, follow-ups get forgotten, and the real blocker (missing PO, invoice not received, next payment run) surfaces weeks late.

Promise: "Tell Claude who owes you. Tagado follows up in your voice, reacts when they reply, and stops the moment you confirm payment."

The one rule that decides every design question:

Who	Does	Never does
Claude (via MCP)	Reads invoices and client history, drafts and redrafts messages, answers "who owes me / what needs attention", records a payment the owner confirms in chat	Sends anything, approves plans, resolves issues
Tagado backend	Stores, validates, schedules, sends approved messages, reacts to client responses with fixed rules, computes client history	Writes text with AI, guesses intent, sends unapproved messages
Owner (human)	Approves plans, confirms payments, resolves issues, resumes paused plans	Has to remember dates

Who	Does	Never does
Client payer	Opens the link, pays, reports payment, gives a date, raises a need or problem	Needs an account or Claude

v1 is: invoice tracking, a drafted follow-up plan per invoice, scheduled email sending, a public client response page, an owner dashboard, and 8 MCP tools.

v1 is not: an invoicing/GST tool, a payment gateway, WhatsApp, bank reconciliation, partial-payment accounting, multi-user teams, or anything with AI inside the app.

2. A real use case, end to end

This story is the product. Every screen, tool and rule in this spec exists to make one of its steps work.

The business. Priya runs Pixelcraft Studio, a 6-person design agency in Chennai. Zenith Logistics owes her ₹2,36,000 (incl. GST) for a brand refresh, invoice PX-2041, due 30 Nov. Zenith has paid late before: 38 days late on average, 2 broken promises. Anita Rao is her contact; payments go through Zenith's accounts team. Priya hates chasing Anita because Zenith is her second-biggest client.

What Priya types in Claude (with the invoice PDF attached)

> Track this invoice for Zenith. ₹2,36,000 incl. GST, due 30 Nov. Contact Anita Rao, anita@zenithlogistics.in, CC ap@zenithlogistics.in.

What Claude does

1. Reads the PDF: invoice number PX-2041, amount, issue and due dates, short description.
2. Calls `get_client_context("Zenith")`. Tagado returns: 4 past invoices, 25% on time, 38 days late on average, 2 broken promises, Priya's note "Always says 'processing'. Be polite but specific about dates.", and today's date in IST.
3. Drafts a plan that is polite but date-specific: 27 Nov heads-up asking to confirm it is in the next payment run; 1 Dec due-date note asking for the expected payment date; 8 Dec names the delay and asks for a date by Friday; 15 Dec final, firm request for a committed date. Plus a promise check-in, used only if Zenith commits to a date.
4. Calls `create_invoice_with_plan` and gets back a review link.

5. Replies: “I’ve set up PX-2041 for Zenith with 4 follow-ups. Zenith has paid late before and missed two promised dates, so these ask for specific dates. Nothing is sent until you approve: [link].”

What happens in Tagado, and after the chat ends

Date	What happens	Who acts
24 Nov	Priya opens the link, softens one line, turns on the MSME note for the 15 Dec message, clicks Approve	Priya, 2 min
27 Nov, 10:30 IST	Reminder 1 emailed to Anita, CC accounts, with a “View invoice and respond” button	Tagado
3 Dec	Zenith accounts opens the page, picks “I need something → PO number missing”. All reminders held instantly; Priya gets an email	Client → Tagado
3 Dec	Priya replies to their email with the PO, clicks Resolve → Resume	Priya, 1 min
8 Dec	Accounts picks “Payment is scheduled → 15 Dec”. Reminders stay held; check-in set for 16 Dec	Client → Tagado
15 Dec	Accounts reports payment with UTR. Priya checks her bank, clicks Confirm. Invoice paid, everything stops	Client → Priya
Next month	Zenith’s history now shows a kept promise; Claude’s next drafts for Zenith adapt	Tagado → Claude

The contrast that proves “relationship-aware”

Priya then types: “Also track PX-2042 for Acme, ₹1,80,000, due 10 Dec.” Acme’s context shows 5 of 6 invoices paid on time and the note “Client since 2021, keep it warm”. Claude drafts three short, friendly messages with no date pressure. Same request shape, visibly different plans, because the history lives in Tagado.

Why this needs Tagado, not just Claude or Zoho

- Claude alone can write the messages but cannot send on 27 Nov at 10:30, notice a click on 3 Dec, or hold reminders while Priya fixes the PO.
- Zoho or Tally send the same template to every client and would have kept sending “overdue” to a team waiting on a PO.

- Priya spent about 3 minutes in three weeks, never wrote an awkward email, and nothing left without her approval. Zenith never needed an account or Claude.

3. How Tagado works

One FastAPI process exposes four doors (MCP, the owner API, the public client API, the cron tick), and all four call the same service layer. That single rule is what keeps behaviour identical whether an action comes from Claude, the dashboard, a client click or the scheduler.



How a request flows:

1. **Claude → MCP.** Claude.ai's servers (not the owner's laptop) call `https://<host>/mcp/?key=<workspace token>`. Middleware resolves the token to a workspace and stores it in a request context variable. The tool validates input with Pydantic, calls a service function, and returns JSON plus a one-line summary.

2. **Owner → React → API.** The React app is built to static files and served by the same FastAPI app, so the session cookie works with no CORS setup. Every owner route reads the workspace from the session and calls services.
3. **Client → public page.** `/p/:token` is a React route; it calls `/api/public/invoices/{token}`. Responses call `engine.client_*` functions.
4. **Scheduler → cron tick.** An external cron calls `/api/cron/tick`, which claims due messages atomically and sends them. It also runs time-based checks (broken promises, completed plans).

What lives where: text Claude wrote lives in `messages`; history lives in `events`; client stats are computed from `invoices` on demand; attention items are a query, not a table.

4. Design decisions and why

Each decision below fixes a flaw found while pressure-testing earlier versions of this plan. Cursor should not “simplify” any of them away.

Naive assumption	Why it breaks	v1 decision
The client pays by scanning a UPI QR	UPI has per-transaction limits (commonly ₹1 lakh); B2B teams pay by NEFT/RTGS in payment runs	Bank details with copy buttons first. UPI QR only if the owner set a UPI ID and outstanding ⩽ ₹1,00,000
The email reaches the person who pays	It often reaches a project contact; delays come from missing PO/GSTIN, invoice not received, approvals	Structured responses built around real AP reasons; “Share with accounts team” on the page; CC support
“I’ve paid” means paid	A click is a claim	“Payment reported” holds reminders; only the owner confirms. No auto-resume timeout, ever
Claude can write legal wording	Free-form legal text is a liability	Claude never writes legal or tax text. The app appends one fixed MSME paragraph, only with a saved Udyam number and an owner toggle
Claude remembers how clients pay	Claude has no memory between chats	Tagado computes history and serves it via `get_client_context` before drafting

Naive assumption	Why it breaks	v1 decision
Messages written on day 0 stay valid	They can't know about a promise on day 33	Any client response holds the remaining messages; a promise triggers one pre-written check-in; everything else goes to the owner
Claude edits are safe after approval	Then Claude could change what is sent without review	Any MCP edit sets the plan back to draft; only the owner approves in the app

5. Users and roles

Three people touch Tagado, and only one of them logs in.

Role	Example	Where they act	Auth
Owner	Priya, Pixelcraft Studio	Claude (via MCP) and the Tagado dashboard	Email + password (session cookie); MCP via a secret per-workspace URL
Client payer	Anita at Zenith, or Zenith's accounts team	Email inbox and the public invoice page	None; a 32-character random token in the link
Reviewer (Volopay)	Testing your submission	Same as the owner, on the seeded demo workspace	Test login + demo MCP URL from the submission email

Design for the reviewer explicitly. They will connect their own Claude, create an invoice using their own email as the client email, and expect the reminder to arrive. So "Send now" must exist, email must reach any address, and the demo workspace needs a one-click Reset.

6. User flows

Seven flows cover the whole product. A–C happen once per invoice; D–G are where Tagado earns its keep after the chat ends.

A. Setup (once, about 3 minutes)

1. Owner logs in, opens **Settings**, fills business name, sender name, reply-to email, bank details (account name, number, IFSC), optional UPI ID, optional Udyam number.
2. Settings shows the workspace's MCP URL with Copy and the connect steps.
3. In Claude: Settings → Connectors → Add custom connector → name "Tagado", paste URL → Add → Connect. Then ask "What can Tagado do?" to confirm.

B. Create an invoice and plan (in Claude)

1. Owner asks Claude to track an invoice (typed details or an attached PDF).
2. Claude calls `get_client_context` → history and notes, or `found: false`.
3. Claude drafts 3–5 messages plus a promise check-in and calls `create_invoice_with_plan`.
4. Backend validates, creates or updates the client (matched by email), saves the invoice as `open` and the plan as `draft`, returns `review_url`.
5. Claude replies with a short summary and the link, stating nothing is sent until approval.

C. Review and approve (in the app)

1. Owner opens `review_url` → invoice page shows each message as an email preview with its send date and time.
2. Owner edits wording or dates, optionally toggles the MSME note on the last message (enabled only if a Udyam number is saved).
3. Owner clicks **Approve plan** → plan `active`, messages `scheduled`. Banner: "Tagado sends these at 10:30 AM IST on the dates shown and pauses automatically if the client responds."

D. Sending (automatic)

1. Every 5 minutes the cron tick sends due messages for active plans on unpaid invoices.
2. Each email = Claude's text + an app-made footer (outstanding amount, due date, "View invoice and respond" button). Reply-To is the owner.
3. Every send and every client page view is logged on the invoice timeline.

E. Client responds (public page, no login)

Four choices. The first three hold all remaining scheduled messages immediately, then:

1. **I've paid** (amount, date, UTR/reference) → `payment_reported` → owner emailed → held until the owner acts.

2. **Payment is scheduled** (date, today to +30 days) → `promised` → promise check-in scheduled for the day after → owner emailed.
3. **I need something** (invoice copy, PO number, GSTIN correction) or **There's a problem** (amount, work incomplete, other) + note → `issue_raised`, plan `paused` → owner emailed.
4. **Share with accounts team** → copies the link. No state change.

On revisit the page shows the current state ("Payment reported on 12 Nov — awaiting confirmation from Pixelcraft"), never the same form twice.

F. Owner handles it (app or Claude)

Situation	Owner action	Result
Payment reported	Confirm received	`paid`; remaining messages cancelled; client page shows "Paid — thank you"
Payment reported	Not received yet	Back to `open`; held messages rescheduled from the next slot, keeping gaps; owner should reply to the client
Promised; check-in sent; unpaid 3 days after promise date	Resume plan, or redraft via Claude	Promise marked broken in history; plan paused until owner chooses
Issue raised	Resolve → Resume, or Cancel invoice, or edit amount and redraft via Claude	Resume reschedules; redraft returns plan to `draft`
Sequence ended, unpaid	Ask Claude for next steps, or add messages	Listed in Needs attention

In Claude, "What needs my attention?" calls `get_attention_items`; Claude summarises and drafts a reply the owner sends from their own email, or new messages via `replace_remaining_messages`.

G. Ask anytime (Claude)

"Who owes me money?" · "How much is overdue?" · "How does Zenith usually pay?" · "Acme paid, I checked the bank" (→ `record_payment`). Claude reads; the owner decides.

7. State machine

Three status fields drive everything. All transitions live in `backend/app/services/engine.py`; MCP tools, API routes, the public page and cron all call these functions. No other file may write a status field. Each function runs in one DB transaction and writes an `events` row.

Invoice status: `open` · `promised` · `payment_reported` · `issue_raised` · `paid` · `cancelled`. "Overdue" is derived for display (`open` or `promised`, and due date < today IST), never stored.

Plan status (column on invoice): `draft` · `active` · `paused` · `completed` · `cancelled`.

Message status: `draft` · `scheduled` · `held` · `sending` · `sent` · `failed` · `cancelled`. Message `kind`: `main` or `promise_checkin`.

Send rule (the only one): cron sends a message only when message = `scheduled`, `send_at` ≤ now, plan = `active`, invoice is `open` or `promised`.

Function	Allowed from	Invoice →	Plan →	Messages	Also
`create_invoice_with_plan`	—	`open`	`draft`	all `draft`; check-in `draft`, no `send_at`	event `created_by_claude`
`approve_plan`	plan `draft`; reply-to set	—	`active`	main `draft` → `scheduled`; past `send_at` → next slot	event
`pause_plan` / `resume_plan`	`active` / `paused`	—	`paused` / `active`	`scheduled` ↔ `held`; resume re-spaces from next slot keeping gaps (min 2 days)	event
`client_reported_payment`	`open`, `promised`	`payment_reported`	unchanged	all `scheduled` → `held`	store amount, date, UTR; notify owner
`client_promised(date)`	`open`, `promised`	`promised`	unchanged	main `scheduled` → `held`; check-in → `scheduled` at date + 1 day	`promise_count` += 1; notify owner
`client_raised_issue`	`open`, `promised`, `payment_reported`	`issue_raised`	`paused`	all `scheduled` → `held`	store reason + note; notify owner
`mark_promise_broken` (cron)	`promised`, check-in `sent`, today > promise date + 3	`open`	`paused`	—	`promise_broken` = true; attention
`confirm_payment`	not `paid` / `cancelled`	`paid`	`completed`	unsent → `cancelled`	`paid_at` = now
`reject_payment_report`	`payment_reported`	`open`	`active`	`held` → re-spaced `scheduled`	event
`resolve_issue_and_resume`	`issue_raised`	`open`	`active`	`held` → re-spaced `scheduled`	event
`replace_remaining_messages`	not `paid` / `cancelled`	—	`draft`	unsent → `cancelled`; new ones `draft`	needs re-approval

Function	Allowed from	Invoice →	Plan →	Messages	Also
`update_message` (via MCP)	message unsent	—	`draft`	edited message `draft`; other `scheduled` → `draft`	needs re-approval
`update_message` (owner in app)	message `draft` / `scheduled` / `held`	—	unchanged	status unchanged	owner is the approver
`cancel_invoice`	not `paid`	`cancelled`	`cancelled`	unsent → `cancelled`	—
Last main message sent, unpaid (cron)	plan `active`	—	`completed`	—	attention: sequence ended

Two deliberate rules: a client action can never resume sending, and `payment_reported` never times out. Only the owner restarts reminders.

8. Screens (React routes)

Seven routes: six for the owner and one public page for clients. The dashboard leads with what needs action, because the owner opens Tagado to act, not to browse.

Route	Screen	Auth	Priority
`/login`	Login	Public	P0
`/`	Dashboard	Owner	P0
`/invoices/:id`	Invoice detail (review, approve, handle responses)	Owner	P0
`/settings`	Settings + Connect Claude	Owner	P0
`/clients` and `/clients/:id`	Clients list and detail	Owner	P1 (seed notes in P0)
`/p/:token`	Client payment and response page	Public (token)	P0

8.1 Login

Email + password. Under the form: “Reviewer? Use the demo login from the submission email.” Sign-up is P1.

8.2 Dashboard

1. **Top strip, 4 numbers:** Outstanding (₹, count) · Overdue (₹, count) · Needs attention (count) · Collected in last 30 days (₹).

2. **Needs attention list** (hidden when empty). Row: client, invoice #, reason, how long ago, one primary button. Reasons in priority order: Issue raised · Payment reported (verify) · Promise broken · Plan awaiting approval · Sequence ended unpaid · Send failed.
3. **Invoices table:** Client · Invoice # · Amount · Due · Days overdue (red if > 0) · Status chip · Next reminder (date, "Held" or "—"). Filter tabs: All · Open · Overdue · Promised · Issues · Paid. Row click → invoice detail.
4. **Empty state:** "No invoices yet. In Claude, say: 'Track my invoice to Acme for ₹50,000 due 15 Dec.'" + link to Settings → Connect Claude.

8.3 Invoice detail

- **Header:** client, invoice #, amount, outstanding, due date, status chip, "Created by Claude" tag. Buttons change with state: Approve plan · Pause · Resume · Confirm payment · Not received · Resolve & resume · Cancel invoice · Copy client link · Open client page.
- **Draft banner** while plan is `draft`: "Nothing has been sent. Review the messages and approve."
- **Client response card** (when `payment_reported`, `promised` or `issue_raised`): what the client said, when, and the buttons that apply.
- **Left: Follow-up plan.** One card per message in order: step, tone, send date/time, status chip, subject, body. Unsent cards: Edit inline (subject, body, date), Delete, **Send now**, **Preview email** (rendered by the backend's compose function). Last main card: "Include MSME note" toggle, disabled with a tooltip if no Udyam number. The check-in card sits at the bottom: "Sent only if the client promises a date."
- **Right: Timeline.** Every event, newest first: created by Claude, approved, sent (to whom), client viewed page, client responses, owner actions, failures.

8.4 Clients

List: name, contact, outstanding, invoices, on-time %, average days late, broken promises. Detail: same stats, editable relationship notes (Claude reads these), contact email, CC emails, the client's invoices. The stats are exactly what `get_client_context` returns, so the owner sees what Claude sees.

8.5 Settings

Business name · sender name · reply-to email · bank account name / number / IFSC · UPI ID (optional) · Udyam number (optional, format `UDYAM-XX-00-0000000`) · read-only preview of the MSME paragraph labelled "template, pending CA review" · **Connect Claude** box:

MCP URL with Copy, the 3 connect steps, Regenerate URL · **Reset demo data** (demo workspace only).

8.6 Client page `/p/:token` (mobile-first)

1. Header: “Payment request from Pixelcraft Studio”.
2. Invoice card: invoice #, description, amount, outstanding, due date, “View invoice” if the owner gave a URL.
3. Status line in plain words: “Due in 5 days” · “12 days overdue” · “Payment reported, awaiting confirmation” · “Paid — thank you” · “This request was withdrawn”.
4. **How to pay:** bank details with Copy buttons and “Please quote PX-2041 as reference”. UPI QR only when allowed (section 4).
5. **Respond:** four large buttons (section 6E), each opening an inline form of at most 3 fields.
6. Footer: “Questions? Reply to the email or write to {reply-to}.” · “Sent via Tagado”.

No login or cookies. Log `client_viewed` on load, at most once per hour per invoice.

9. MCP server: how it is built and connected

The MCP server is not a separate product; it is a fourth door into the same FastAPI app and the same services. Tools contain no business logic of their own.

9.1 Building it

- **Library:** the official MCP Python SDK (`pip install mcp`), using `FastMCP` . This is not an AI library; it only speaks the protocol.
- **Transport:** Streamable HTTP, stateless mode, JSON responses. No stdio, no SSE.
- **Mounted inside FastAPI** at `/mcp` . The SDK’s session manager must be started in FastAPI’s lifespan, or every call fails.
- **Auth:** a per-workspace secret in the URL query (`?key=`). An ASGI middleware in front of the MCP app looks up `workspaces.mcp_token` , returns 401 if not found, and sets a `contextvars.ContextVar` with the workspace id. Every tool reads the workspace from that variable, never from tool input.
- **Tool pattern:** each tool is a thin function: Pydantic-validate input → call one service function → return a dict (JSON) that includes a short `summary` string and, where relevant, an absolute `review_url` (`APP_URL/invoices/{id}`). Validation errors are raised as tool errors with one sentence naming the field and the fix, so Claude can retry.

- **Every read tool returns `today_ist`** (YYYY-MM-DD). Claude has no reliable clock; this stops it scheduling in the past or the wrong year.
- **Annotations:** mark read tools `readOnlyHint: true` so Claude can treat them as safe.

Indicative skeleton (verify against the installed SDK version's README; names move between releases):

```
# backend/app/mcp/server.py
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("Tagado", instructions=SERVER_INSTRUCTIONS,
stateless_http=True, json_response=True)
mcp.settings.streamable_http_path = "/"          # so the endpoint is exactly
/mcp/

@mcp.tool()
def get_client_context(query: str) -> dict:
    ws = current_workspace_id.get()               # set by the token
middleware
    return services.clients.get_context(ws, query)

# backend/app/main.py
@asynccontextmanager
async def lifespan(app):
    async with mcp.session_manager.run():
        yield
app = FastAPI(lifespan=lifespan)
app.include_router(api_router, prefix="/api")
app.mount("/mcp", McpTokenMiddleware(mcp.streamable_http_app()))
app.mount("/", SpaStaticFiles(directory="frontend_dist", html=True))   # LAST
```

9.2 Connecting it to Claude

1. Deploy first: Claude.ai connects from Anthropic's servers, so `localhost` never works. For local testing, use a tunnel (cloudflared or ngrok).
2. Test with MCP Inspector: `npx @modelcontextprotocol/inspector`, transport "Streamable HTTP", URL `https://<host>/mcp/?key=<token>`, call every tool.
3. In Claude.ai: Settings → Connectors → Add custom connector → Name "Tagado", URL as above → Add → Connect. No OAuth fields.
4. In a new chat, check the Tagado tools are enabled, then ask "What can Tagado do?". Choose "Always allow" for read tools when prompted.

9.3 Connecting it to the product

The same row Claude creates is what the React app shows, because both go through `services/`. A Claude-created invoice appears on the dashboard on next load; an owner edit in the app is what `get_invoice` returns to Claude. Events record `actor = claude` or `owner`, so the timeline shows who did what.

9.4 Known pitfalls (check these first if Claude can't connect)

- **Trailing slash redirect:** a Starlette mount can 307-redirect `/mcp` → `/mcp/`; some clients drop the POST body on redirect. Publish the URL with the trailing slash, exactly as tested in Inspector.
- **Host header protection:** newer SDK versions can reject unknown `Host` headers (e.g. 421). Configure the SDK's transport-security allowed hosts to include your deployed domain.
- **SPA catch-all order:** the static/SPA mount must be registered last, or it swallows `/api` and `/mcp`.
- **Cold starts:** a sleeping free-tier server can make Claude time out on connect. The 5-minute cron ping keeps it warm; open the app once before demoing.

10. MCP tools, one by one

Eight tools: four read, four write. Claude can read everything and draft anything, but nothing it creates or edits is sent until the owner approves it in the app.

Server instructions (MCP `instructions` field, verbatim)

> Tagado tracks invoices and sends follow-up emails on the owner's behalf. Workflow: (1) call get_client_context before drafting for any client; (2) match tone to history and owner notes: warm for reliable long-term clients, clear and specific about dates for clients with late payments or broken promises, never threatening; (3) call create_invoice_with_plan; (4) give the owner the review_url and say nothing is sent until they approve in Tagado. Use today_ist from tool results for all dates. Never include bank details, payment links, or legal or tax references in message text; Tagado adds payment details and any legal note itself. Write in the owner's voice, first person, short paragraphs, Indian English. Use placeholders for amounts and dates.

Message format (shared by the write tools)

- `messages[]`: 1–6 items, each `{ send_on: "YYYY-MM-DD", tone: "friendly" | "neutral" | "firm", subject: ≤120 chars, body: ≤1,500 chars plain text }`.

- `promise_checkin`: `{ subject, body }`.
- Allowed placeholders: `{{contact_name}}`, `{{invoice_number}}`, `{{amount}}`, `{{outstanding}}`, `{{due_date}}`, `{{business_name}}`; `{{promise_date}}` only in the check-in. Any other `{{…}}` → error listing the allowed ones.
- Validation: `send_on` ascending, ≥2 days apart, first ≥ today IST, last ≤ due date + 60. Sundays shift to Monday (reported in the result, not an error).

1. `get_client_context` (read)

- **Purpose:** give Claude the relationship history before it drafts.
- **When:** before any create or redraft; when the owner asks how a client pays.
- **Input:** `query` (client name or email; case-insensitive partial match).
- **Does:** `services.clients.get_context()` → matches up to 3 clients, computes stats (section 12).
- **Returns:** `today_ist`, `found`, and per match: `client_id`, name, contact, email, CC, `relationship_notes`, stats `{invoices_total, paid_count, on_time_pct, avg_days_late, broken_promises, open_issues, outstanding}`, last 5 invoices (number, amount, due, paid date, days late).
- **Errors:** none; `found: false` with "New client; create_invoice_with_plan will create them."

2. `create_invoice_with_plan` (write)

- **Purpose:** track an invoice and save a draft plan in one atomic call.
- **Description (verbatim):** "Track an invoice and save a draft follow-up plan. Write 3–5 messages: one a few days before the due date, one on or just after it, then spaced follow-ups that get gradually clearer, ending with a final message that asks for a payment date. Also write promise_checkin, sent only if the client commits to a date: a polite check-in referencing {{promise_date}}. Use placeholders instead of typing amounts and dates. Saved as draft; the owner approves in Tagado."
- **Input:** `client_id? + client {name, contact_name, email, cc_emails?, notes?}`, `invoice {invoice_number, amount_inr, issue_date, due_date, description?, invoice_url?}`, `messages[]`, `promise_checkin`.
- **Does:** `services.plans.create_invoice_with_plan()` in one transaction: if client_id (from get_client_context) is given, use that client, else upsert by email (notes only set if empty) — this stops a typo'd email from creating a duplicate client with no history, create invoice `open` + `public_token`, validate and store messages `draft`, event `created_by_claude`.

- **Returns:** `invoice_id`, `review_url`, `client_page_url`, messages with computed `send_at` (IST), any Sunday shifts, client stats, `summary`.
- **Errors:** invalid email or dates; unknown placeholder; duplicate `invoice_number` → returns the existing invoice's id and `review_url` with "Already tracked; use replace_remaining_messages to change its plan." (makes retries safe).

3. `update_message` (write)

- **Purpose:** change one unsent message ("make the second one softer").
- **Input:** `message_id`, optional `subject`, `body`, `send_on`.
- **Does:** validates, updates; sets plan to `draft` and all its `scheduled` messages to `draft` (section 7). Event `edited_by_claude`.
- **Returns:** updated message, `review_url`, `summary` stating re-approval is needed.
- **Errors:** message sent/cancelled/sending; invoice paid/cancelled.

4. `replace_remaining_messages` (write)

- **Purpose:** redraft after a broken promise, a resolved issue, or a change of approach.
- **Input:** `invoice_id`, `messages[]`, optional `promise_checkin`.
- **Does:** cancels unsent messages, inserts new ones as `draft`, plan → `draft`.
- **Returns:** new messages, `review_url`, `summary`.
- **Errors:** invoice paid or cancelled; validation as above.

5. `list_invoices` (read)

- **Input:** optional `status` (`all` default, `open`, `overdue`, `promised`, `issues`, `payment_reported`, `paid`), optional `client`.
- **Returns:** `today_ist`, totals (outstanding, overdue), rows: id, client, number, amount, outstanding, due, days overdue, status, plan status, next send. Max 50 rows.

6. `get_invoice` (read)

- **Input:** `invoice_id` or `invoice_number`.
- **Returns:** invoice fields, client, all messages (id, kind, status, send_at, subject, body), client response details, last 20 events, `review_url`, `client_page_url`, `today_ist`.

7. `get_attention_items` (read)

- **Purpose:** answer "what needs my attention?"

- **Returns:** `today_ist` and items in priority order, each: `type`, invoice id/number, client, what happened (e.g. “Zenith accounts: ‘PO number missing’, 3 Dec”), and `owner_options` (e.g. “Resolve and resume in Tagado”, “Ask me to redraft”).

8. `record_payment` (write)

- **Purpose:** the owner confirms in chat (“Acme paid, I checked the bank”).
- **Description note:** “Only call when the owner explicitly confirms the money was received. Never infer payment.”
- **Input:** `invoice_id`, optional `reference`, optional `note`.
- **Does:** `engine.confirm_payment(actor="claude_on_owner_instruction")`; full amount only in v1.
- **Returns:** confirmation, cancelled message count, `summary`.

Deliberately not tools: approve plan, resume plan, resolve issue, send now. These stay human-only in the app.

11. Backend REST API (for the React app)

All owner routes require the session cookie and scope every query by the session’s `workspace_id`. Every state-changing route calls exactly one `engine` function and returns the updated invoice (with messages and the last events), so the React page re-renders from one response.

Method & path	Purpose	Calls
`POST /api/auth/login` · `POST /api/auth/logout` · `GET /api/auth/me`	Session	`services.auth`
`GET /api/dashboard`	4 totals + attention items	`stats`, `attention`
`GET /api/invoices?status=&client=`	Invoices table	`invoices.list`
`GET /api/invoices/{id}`	Detail: invoice, client, messages, events	`invoices.get`
`PATCH /api/invoices/{id}`	Edit amount, due date (`shift_reminders: bool`), description, invoice_url	`engine.update_invoice`
`POST /api/invoices/{id}/approve` · `/pause` · `/resume` · `/cancel`	Plan actions	`engine.*`
`POST /api/invoices/{id}/confirm-payment` · `/reject-payment` · `/resolve-issue`	Response handling	`engine.*`
`PATCH /api/messages/{id}` · `DELETE /api/messages/{id}`	Edit or delete unsent message (owner)	`engine.update_message(actor=owner)`
`POST /api/messages/{id}/send-now`	Immediate send (plan must be active)	`email.send_one`

Method & path	Purpose	Calls
`GET /api/messages/{id}/preview`	Rendered HTML + subject exactly as sent	`email.compose`
`GET /api/clients` · `GET /api/clients/{id}` · `PATCH /api/clients/{id}`	Clients and notes	`clients.*`
`GET /api/settings` · `PATCH /api/settings`	Workspace settings	`workspaces.*`
`POST /api/settings/regenerate-mcp-token`	New MCP URL	`workspaces.rotate_token`
`POST /api/demo/reset`	Re-seed (only if `is_demo`)	`seed.reset_demo`
`GET /api/public/invoices/{token}`	Client page data (no owner-private fields)	`public.get` + log view
`POST /api/public/invoices/{token}/respond`	`{type: paid | scheduled | need | problem, ...}`	`engine.client_*`
`GET /api/public/invoices/{token}/upi-qr.png`	QR image when allowed	`qr.render`
`POST /api/cron/tick`	Scheduler (header `x-cron-secret`)	`scheduler.tick`
`GET /api/health`	Uptime ping	—

Public payload contains only: business name, reply-to email, invoice number, description, amounts, dates, status, bank details, UPI availability, invoice URL. Never other invoices, client stats, messages or notes.

12. Data model

Six tables (SQLAlchemy 2.0 models + Alembic). Money is integer paise; `date` columns are calendar dates; timestamps are `timestamptz` in UTC and converted to IST (`zoneinfo("Asia/Kolkata")`) only for display and scheduling. Attention items are a query, not a table.

```
users        id uuid PK · email unique · password_hash · name · workspace_id
FK

workspaces   id · business_name · sender_name · reply_to_email
             bank_account_name? · bank_account_number? · ifsc? · upi_id? ·
udyam_number?
             mcp_token unique (32+ random url-safe chars) · send_hour_ist=10
· send_minute_ist=30
             is_demo bool · created_at

clients      id · workspace_id FK · name · contact_name · email · cc_emails
text[] · relationship_notes?
             UNIQUE(workspace_id, email)

invoices     id · workspace_id FK · client_id FK · invoice_number ·
```

```
description? · invoice_url?
             amount_paise int · amount_paid_paise int=0 · issue_date date ·
due_date date
             status text · plan_status text · public_token unique (32 random
chars)
             promise_date date? · promise_count int=0 · promise_broken
bool=false
             reported_at? · reported_amount_paise? · reported_reference? ·
reported_paid_on date?
             issue_kind? (need|problem) · issue_reason? · issue_note?
             paid_at? · created_at · UNIQUE(workspace_id, invoice_number)

messages     id · invoice_id FK · kind (main|promise_checkin) · step int ·
tone · subject · body
             send_at timestamptz? (null for check-in until a promise) ·
status text
             include_msme_note bool=false · sent_at? · sent_to? · error? ·
attempts int=0
             INDEX(status, send_at)

events       id · invoice_id FK · actor (owner|client|claude|system) · type ·
data jsonb? · created_at
             INDEX(invoice_id, created_at)
```

Client stats (`services/stats.py` , used by the Clients page and `get_client_context`): total invoices; paid count; on-time % = paid on or before due date ÷ paid; average days late over paid invoices (0 if early); broken promises = invoices with `promise_broken` ; open issues; outstanding total.

Attention query (`services/attention.py`), in priority order: status `issue_raised` · status `payment_reported` · `promise_broken` and plan `paused` · plan `draft` · plan `completed` and status `open` (sequence ended) · any message `failed` .

13. Scheduler and email engine

The scheduler is one endpoint, `POST /api/cron/tick` , called every 5 minutes by cron-job.org with header `x-cron-secret` . It must be safe if two ticks run at the same moment. Do not use an in-process scheduler (APScheduler, Celery beat): a sleeping or multi-worker server would skip or duplicate sends.

Each tick (`services/scheduler.py`)

1. Reject with 401 unless the secret matches.

2. **Claim atomically:** `UPDATE messages SET status='sending', attempts=attempts+1 WHERE id IN (SELECT m.id FROM messages m JOIN invoices i ... WHERE <send rule> ORDER BY send_at LIMIT 20 FOR UPDATE SKIP LOCKED) RETURNING *`. Only rows this tick flipped get sent.
3. Send each claimed message. Success → `sent`, `sent_at`, `sent_to`, event `message_sent`. Failure → back to `scheduled` if attempts < 3, else `failed` (shows in attention).
4. Recover stuck rows: `sending` for more than 10 minutes → `failed`.
5. Promise checks: invoices `promised` whose check-in is `sent` and today IST > promise date + 3 days → `engine.mark_promise_broken`.
6. Plans `active` with no unsent main messages and invoice still `open` → `completed`.
7. Return counts as JSON.

Send now calls the same `send_one` for one message: it skips the time and window checks but never the state checks (plan `active`, invoice `open` / `promised`, message `scheduled` / `held` → claimed as `sending` first).

Send-time rules (`services/plans.py`)

- `send_on` date → `send_at` = that date at the workspace send time (default 10:30 IST), stored as UTC.
- Scheduled sends only Monday–Saturday, 9:00–19:00 IST. Sundays shift to Monday.
- On approve and resume, any past `send_at` moves to the next valid slot.
- **Re-spacing** on resume: keep the gaps between consecutive held messages (min 2 days), starting at the next valid slot.

Composing an email (`services/email/compose.py`, Jinja2)

- **From:** `"{sender_name} via Tagado" <EMAIL_FROM>` · **Reply-To:** workspace reply-to · **To:** client email · **CC:** client CC emails.
- **Subject and body:** Claude's text with placeholders filled; amounts Indian format (₹2,36,000); dates "30 Nov 2026". Body paragraphs from blank lines; escape HTML.
- **Footer (always, app-made):** outstanding amount, due date, button "View invoice and respond" → `APP_URL/p/{public_token}`, "Sent on behalf of {business_name}".
- **MSME note** (only if `include_msme_note` and a Udyam number exists), fixed text: "{business_name} is registered as a micro/small enterprise (Udyam {number}). Under the MSMED Act, payments to registered micro and small enterprises are due within the agreed period, not exceeding 45 days." Mark it "template, pending CA review".
- Send HTML and plain text. The preview endpoint uses the same function, so what the owner approves is byte-for-byte what is sent.

Sending (`services/email/send.py`)

One function `send_email(to, cc, subject, html, text, reply_to)` over SMTP (`smtplib` with STARTTLS, Gmail app password). Keep it behind this interface so switching to Resend later is one file.

Owner notifications

On any client response, email the reply-to address: “Zenith reported payment of ₹2,36,000 (UTR …). Confirm in Tagado: {link}.” Same send function, no CC.

14. Edge cases and guardrails

These are the cases a reviewer is most likely to poke at, each with its decided behaviour.

Case	Behaviour
Claude retries `create_invoice_with_plan`	Unique invoice number per workspace → returns the existing invoice; no duplicate
Claude edits a message after approval	Plan returns to `draft`; nothing sends until re-approved
Claude proposes a date in the past or wrong year	Rejected with “today_ist is 2026-11-10; first send_on must be on or after it”
Send now on a draft plan	Button disabled: “Approve the plan first”
Owner edits a message while cron is sending it	Message in `sending` → edit rejected with “Being sent right now”
Client promises again while `promised`	Allowed; check-in rescheduled; `promise_count` ⩾ 2 shows “2nd extension” in attention
Promise date in the past or > 30 days ahead	Rejected on the page with a clear message
Client double-taps a response	Button disabled on submit; server ignores the same response type within 60 seconds
Client page for a paid or cancelled invoice	“Paid — thank you” or “This request was withdrawn”; no forms
Owner changes the due date	Prompt “Shift unsent reminders by N days?”; yes moves them

Case	Behaviour
Owner changes the amount	Placeholders pick up the new outstanding; if any message was `scheduled`, plan returns to `draft`
Email bounces or SMTP fails	Retries up to 3 ticks, then `failed` + attention “Email to … failed — check the address”
Token guessing	Client pages only by 32-char random token; owner routes always filter by `workspace_id`
MCP URL leaks	Settings → Regenerate URL invalidates the old token immediately
Several reviewers on one demo workspace	Reset demo data restores the seed in one click
Reply-to email not set	Approve blocked until set
No bank details set	Client page: “Payment details will be shared by {business_name}”; Approve shows a warning

Hard guardrails enforced in code: at most 6 main messages per invoice; at least 2 days between scheduled sends; scheduled sends only Mon–Sat 9:00–19:00 IST (Send now is an explicit owner action and ignores the window); nothing sends unless the plan is `active`; payment details and legal text come only from the app, never from Claude.

15. Dependencies and loopholes

The build has one critical path: nothing matters until Claude can call a deployed tool and an email can reach an outside inbox. Prove both in the first two hours.

What depends on what

1. **Public HTTPS deploy** → needed for Claude to connect at all (Claude never reaches localhost).
2. **Postgres + migrations + seed** → needed by every tool and screen.
3. **`services/engine.py`** → needed by MCP tools, API routes, public page and cron; build and test it before any UI.
4. **MCP token middleware + one tool working in Claude.ai** → unblocks all other tools.

5. **Email compose + SMTP** → needed by Send now, cron, owner notifications and preview.
6. **Cron tick + cron-job.org** → needed for real scheduling and keeps the free server warm.
7. **React screens** → depend on the API, which depends on services. Build them last among P0.

Loopholes and how they are closed

Loophole	Risk	Closed by
Claude skips `get_client_context`	Generic tone; the core claim fails	Server instructions + tool description; `create_invoice_with_plan` also returns client stats so Claude can redraft
Claude invents dates (wrong year, past)	Messages never send or send wrongly	`today_ist` in every read result; validation errors name today's date
Claude puts bank details or legal threats in text	Wrong account, legal exposure	Instructions forbid it; app footer always supplies payment details; legal text only from app template
Claude edits after approval	Unreviewed text sent	MCP edits force plan back to `draft`
Client clicks "I've paid" falsely	Reminders stop forever	Only holds; owner must confirm; attention item never expires
Duplicate sends (two ticks, retries, Send now during tick)	Embarrassing double emails	Atomic claim with `FOR UPDATE SKIP LOCKED`; `sending` state; Send now claims the same way
Free host sleeps	Cron misses, Claude connect times out	cron-job.org every 5 min hits `/api/cron/tick` (also keeps it awake); open app before demos
Email lands in spam or can't reach reviewers	Demo fails for evaluators	Gmail SMTP app password sends to any address; plain, short emails; test with a non-Gmail inbox
Cookies fail between React and API	Login loops	Serve React from FastAPI (same origin); Vite dev proxy `/api` → `localhost:8000`

Loophole	Risk	Closed by
SPA route swallows `/api` or `/mcp`	404s or HTML instead of JSON	Mount static SPA last; explicit fallback only for non-`/api`, non-`/mcp` paths
MCP URL token visible in logs	Workspace takeover	Regenerate URL; don't log query strings for `/mcp`
Public link forwarded widely	Others see invoice and bank details	Acceptable by design (that is the point); page shows nothing else
Time zone drift	Sends at wrong hour	Store UTC; convert with `Asia/Kolkata` in one helper used everywhere
Custom connectors depend on the reviewer's Claude plan	Reviewer can't connect	Submission email includes the URL, 3 steps, and a 2-min demo video as fallback
AI sneaks in via a dependency	Disqualification	`scripts/check_no_ai.py` fails CI if `requirements.txt` or `package.json` contains `openai`, `anthropic`, `langchain`, `ai`, `@ai-sdk/*`, or any LLM key appears in `.env.example`

16. Tech stack, repo structure, deployment

One Docker image: Node builds the React app, Python serves it together with the API and MCP. One deploy, one URL, same-origin cookies.

Layer	Choice	Notes
Frontend	React + Vite + TypeScript, React Router, TanStack Query, Tailwind (shadcn/ui optional)	Build output copied into the backend image
Backend	Python 3.11+, FastAPI, Uvicorn (1 worker), Pydantic v2	1 worker keeps things simple; cron handles concurrency anyway
DB	Postgres on Neon (free), SQLAlchemy 2.0, Alembic, psycopg 3	Use Neon's pooled connection string
MCP	`mcp` (official Python SDK), FastMCP, stateless Streamable HTTP	Mounted at `/mcp`

Layer	Choice	Notes
Auth	Starlette `SessionMiddleware` (signed cookie) + `bcrypt`	`https_only=True`, `same_site="lax"`
Email	`smtplib` + Gmail app password; Jinja2 templates	Interface allows swapping to Resend + own domain later
QR	`qrcode` (Python) rendering a PNG for `upi://pay?pa=…&pn=…&am=…&tn=…&cu=INR`	Display only
Hosting	Render web service from the Dockerfile	Check current free-tier limits; free instances sleep when idle
Cron	cron-job.org, every 5 min, POST with `x-cron-secret`	Also the keep-alive

Repo structure

```
tagado/
  SPEC.md                      # this document
  Dockerfile                   # stage 1 node: build frontend; stage 2
python: copy dist + backend
  scripts/check_no_ai.py
  frontend/
    src/routes/  Login.tsx Dashboard.tsx InvoiceDetail.tsx Clients.tsx
ClientDetail.tsx Settings.tsx PublicInvoice.tsx
    src/components/  MessageCard.tsx Timeline.tsx AttentionList.tsx
StatusChip.tsx EmailPreview.tsx
    src/api/client.ts          # fetch wrapper, credentials: "include"
    vite.config.ts             # dev proxy /api -> http://localhost:8000
  backend/
    app/main.py                # FastAPI app, lifespan, mounts (/api, /mcp,
SPA last)
    app/config.py              # env vars
    app/db/models.py  app/db/session.py  alembic/
    app/api/  auth.py dashboard.py invoices.py messages.py clients.py
settings.py public.py cron.py
    app/mcp/  server.py tools.py auth_middleware.py
    app/services/  engine.py plans.py stats.py attention.py scheduler.py
clients.py invoices.py
                   workspaces.py seed.py time.py qr.py
    app/services/email/  compose.py send.py templates/
    tests/  test_engine.py test_plans.py test_scheduler.py test_mcp_tools.py
    requirements.txt
```

Environment variables

`DATABASE_URL` · `APP_URL` (e.g. `https://tagado.onrender.com`) · `SESSION_SECRET` · `CRON_SECRET` · `SMTP_HOST=smtp.gmail.com` · `SMTP_PORT=587` · `SMTP_USER` · `SMTP_PASS` (Gmail app password) · `EMAIL_FROM` · `DEMO_EMAIL` · `DEMO_PASSWORD`. No AI keys of any kind.

Deploy steps

1. Create the Neon database; copy the connection string.
2. Render → New Web Service → from repo → Docker; set env vars; deploy.
3. Run `alembic upgrade head` and the seed (a Render shell or a one-off start command).
4. cron-job.org → new job → `POST {APP_URL}/api/cron/tick`, header `x-cron-secret`, every 5 minutes.
5. Log in as demo, copy the MCP URL from Settings, connect it in Claude.

17. Seed and demo data

The seed makes "relationship-aware" visible: two clients with opposite histories, so the same request produces visibly different drafts.

Workspace: Pixelcraft Studio, Chennai · sender "Priya at Pixelcraft" · reply-to = your real email · dummy bank details · UPI ID set · dummy Udyam number in valid format · `is_demo = true`. Login from `DEMO_EMAIL` / `DEMO_PASSWORD`.

Client	Contact	History to seed (all paid, past dates)	Owner's relationship notes
Acme Retail	Rahul Mehta	6 invoices, 5 on time, avg 2 days late, 0 broken promises	"Client since 2021. Rahul is great; their AP is slow at month-end. Keep it warm."
Zenith Logistics	Anita Rao	4 invoices, 1 on time, avg 38 days late, 2 broken promises	"Always says 'processing'. Be polite but specific about dates."
Northwind Foods	Karan Shah	2 invoices, both on time	—

Live invoices so the dashboard isn't empty: a Northwind invoice `open`, plan `active`, next reminder in 3 days · a Northwind invoice `payment_reported` with a UTR · a Zenith invoice `issue_raised` ("Need PO number on invoice") · an Acme invoice `paid` last week.

Acme and Zenith have **no open invoice**, so a reviewer or the demo can create one for each from Claude, live. Client emails in the seed use addresses you control, so test sends never hit real companies.

Reset demo data deletes all rows in the demo workspace and re-runs this seed; it is refused for non-demo workspaces.

18. What to try

These 12 scenarios are the acceptance tests for v1 and the script a reviewer can follow. Run all of them on the deployed URL after a demo reset; every one must pass without touching the database by hand.

#	Where	Do this	Expected result
1	Claude	“What can Tagado do?”	Claude lists the tools; no errors
2	Claude	“Who owes me money?”	`list_invoices` result matching the dashboard totals
3	Claude	“Track invoice PX-2042 for Acme Retail, ₹1,80,000, due 10 Dec. Contact Rahul, <your email>.”	Calls `get_client_context` first; 3 friendly messages; returns review link; dashboard shows it as awaiting approval
4	Claude	Same for Zenith Logistics, PX-2041, ₹2,36,000, due 30 Nov	Drafts are visibly firmer and ask for specific dates; Claude mentions the late history
5	Claude	Repeat request 3 exactly	“Already tracked”; no duplicate invoice
6	App	Open the Zenith review link, edit one line, toggle MSME note on the last message, Preview, Approve	Preview shows footer + MSME paragraph; messages become scheduled at 10:30 IST
7	App	Send now on message 1	Email arrives at your inbox; Reply-To is the owner; button opens the client page; timeline logs the send
8	Client page (phone)	“I need something → PO number”	Status “issue raised”; all scheduled messages held; attention item; owner notification email
9	App	Resolve & resume	Held messages rescheduled from the next slot with gaps kept

#	Where	Do this	Expected result
10	Client page	“Payment is scheduled” → a date 5 days out	Status promised; main messages held; check-in scheduled for the day after
11	Client page, then app	“I’ve paid” with UTR → owner clicks Confirm	Invoice paid; unsent messages cancelled; client page shows thank-you; Zenith stats update
12	Claude	“What needs my attention?” then “Draft a reply to Zenith about the PO”	Lists attention items in priority order; drafts a reply; sends nothing

Safety checks to run too: ask Claude to “send the reminder now” (it must say it can’t; only the app sends); ask Claude to edit an approved message (plan must return to draft); hit `/api/cron/tick` twice in parallel (no duplicate email); open `/p/<random string>` (404).

19. Build order for Cursor

Build riskiest-first. Do not start a phase until the previous phase’s check passes on the deployed URL, not just localhost. Total is about 11 hours.

Phase	Build	Done when	Time
0. Prove the pipe	Repo skeleton, Dockerfile, FastAPI with `/api/health`, MCP mount with one `ping` tool behind the token middleware, Render deploy, Neon DB	MCP Inspector and Claude.ai both call `ping` on the live URL and get the workspace name	1.5 h
1. Core logic	Models, Alembic migration, seed, `time.py`, `plans.py` (validation, send-slot math), `engine.py` (every transition in section 7), `stats.py`, `attention.py`	`pytest` covers every row of the section 7 table and passes	2 h
2. MCP tools	All 8 tools + server instructions + tool descriptions	Scenarios 1–5 in section 18 pass in Claude.ai	1.5 h
3. Email + scheduler	`compose.py`, `send.py`, preview endpoint, Send now, cron tick with atomic claim, owner notifications, cron-job.org	Send now reaches a non-Gmail inbox; a message due in 5 min sends via cron; parallel ticks send once	1.5 h

Phase	Build	Done when	Time
4. Owner app	React: login, dashboard, invoice detail (edit, preview, approve, response actions), settings with Connect Claude	Scenarios 6, 7, 9 pass	2.5 h
5. Client page	`/p/:token` with 4 responses, state-aware views, QR	Scenarios 8, 10, 11 pass on a phone	1 h
6. End-to-end + polish	Demo reset, empty states, loading/error states, `check_no_ai.py`	All 12 scenarios + safety checks pass twice from a clean reset	1 h

How to prompt Cursor

1. Put this doc in the repo root as `SPEC.md` (export as Markdown). First message: “Read SPEC.md fully. Propose an implementation plan following section 19, with the files you’ll create per phase. Don’t write code yet.”
2. Then one phase per chat: “Implement Phase N only, per SPEC.md. Don’t add features, packages or AI libraries not in the spec. List files before writing them.”
3. For Phase 1, insist: “All status changes go through `services/engine.py`, one transaction each, each writing an event. Write pytest tests for every row of the section 7 table.”
4. After each phase: deploy, run that phase’s check on the live URL, commit. If something breaks badly, revert to the last commit rather than patching on top.
5. Paste exact error text and logs back to Cursor; don’t describe them.
6. When Cursor proposes a shortcut that contradicts sections 4, 7 or 14, refuse it; those rules are the product.

The 2-minute video demo this build must support

1. Claude: track PX-2042 for Acme → warm drafts. Track PX-2041 for Zenith → firmer, date-specific drafts.
2. Open Tagado from the link, edit a line, toggle the MSME note, approve.
3. Send now → show the email arriving with the button.
4. Open the client page on a phone → “I need something → PO number” → dashboard shows reminders held and the attention item.
5. Claude: “What needs my attention?” → summary and a drafted reply.

20. Priorities

P0 is about 11 hours. If you fall behind, cut P1 first and never anything on the demo path.

P0 — must ship today

- [ ] Deployed app; MCP connector working in Claude.ai with all 8 tools
- [ ] `engine.py` with every transition, tested
- [ ] Draft → edit → preview → approve flow
- [ ] Send now + cron tick with atomic claiming + cron-job.org
- [ ] Client page with all four responses and state-aware views
- [ ] Pause, resume, confirm payment, reject report, resolve issue, cancel
- [ ] Dashboard with attention list; invoice timeline
- [ ] `get_client_context` stats + seeded clients with opposite histories
- [ ] Settings with MCP URL + connect steps; demo login; Reset demo data
- [ ] `check_no_ai.py` passes

P1 — only after P0 passes all 12 scenarios

- [ ] Clients pages with editable relationship notes
- [ ] Owner email notifications on client responses
- [ ] Due-date change with “shift reminders?” prompt
- [ ] Self-serve sign-up that creates a workspace and MCP URL

P2 — mention in the pitch, don’t build

- WhatsApp reminders (the channel Indian clients actually answer)
- Bank or payment-gateway confirmation to auto-close invoices
- Partial payments and instalment plans
- Import open invoices from Zoho Books or Tally exports
- Team roles (founder approves, ops edits)